Numpy Practice Exercise #1

```
import numpy as np

data=np.random.default_rng().integers(low=0, high=100, size=(8,10))
```

Part 1 Basic indexing

1) 3rd row 5th column

```
data[2,4]
```

2) Entire first row

```
data[0]
```

3) Entire last column

```
data[[0,1,2,3,4,5,6,7],-1]
```

4) Center elements

```
print(np.array([[data[3,4],
                [data[3,5]],
                [data[4,4],
                 data[4,5]]])
)
```

a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	
b_1	b_2	b_3	b_4	b_5	b_6	b_7	b_8	b_9	b_{10}	
c_1	c_2	c_3	c_4	c_5	c_6					
d_1	d_2									
e_1	e_2									
f_1	f_2									
g_1	g_2									
h_1	h_2									

Part 2: Slicing Rows & Columns

5) Slice out rows 2 to 5 & cols 3 to 7

```
data[1:5]    data[:,2:8]
```

or both at once

```
data[1:5,2:8]
```

6) Even numbered rows

```
data[1::2,:]
```

7) Odd-numbered columns

data[:, ::2]

8) Reverse order of rows

data[-1::-1]

9) Reverse order of columns

data[:, -1::-1]

Part 4: Sorting Basics

13) Sort each row of the array independently

np.array([np.sort(i) for i in data])

or

np.sort(data, axis=1) or np.sort(data)

14) Sort each column of the array independently

np.sort(data, axis=0)

15) Sort the entire array as a flat array, then reshape it back to (8,10)

r= data.reshape(-1)

np.sort(r)

print(r.reshape(8,10))

Part 5: Advanced Sorting & Indexing

16) Find the indices that would sort the 4th row

t= np.argsort(data[3])

17) Use those indices to reorder the 4th row manually

data[3] = data[3,t]

18) Sort the array based on the values in the 2nd column

r= data[[i for i in range(8)],1]

s= np.artsort(r)

data = data[s]

Numpy practice exercise #1 continued

19) Get the top 3 largest values from the entire array without fully sorting it

```
cond= data > np.mean(data)
 r  =np.sort(data[cond])
print("top 3 values :", r[[-1,-2,-3]])
```

20) Find the row index where the maximum value of the array is located

```
arr = np.argsort(data)
sr = arr [:,-1]
#row_indices = range(data.shape[0])
# carray = [data[i,j] for i,j in zip(row_indices, sr)]
# carray= np.array (carray)
carray = data [np.arange(data.shape[0]), se]
rle = np.argsort( carray)[-1]
print(" row index where the maximum value of array is
      located", rle)
```

same work of these 3 comments combined

Numpy practice exercise #1 continued

Part 3: Conditional indexing

10) Find all elements greater than 50

```
data[data > 50]
```

11) Create a new array that contains only values between 20 & 80

```
data[(data > 19) & (data < 81)]
```

12) Replace all values less than 30 with -1 (modify the original array)

```
data[data < 30] = -1
```

Part 6: Challenge section

21) For each row, keep only the 3 largest values & set the rest to 0

We created a mask which is False for the largest 3 values in that row & use that mask to update data

```
i1 = np.argsort(data)
i2 = i1[:, 7:]
prem = np.zeros_like(data)
prem[np.arange(8)[:, None], i2] = 1
mask = prem < 1
data[mask] = 0
```

notice how broadcasting rules apply here

22) Create a new array where rows are sorted by their row sum

```
a = np.sum(data, axis=1)
b = np.argsort(a, axis=0)
data[b]
```

23) From the original array, extract a submatrix consisting of rows that have an average value greater than 50

```
data[np.mean(data, axis=1) > 50]
```